

5 MoE环游记：3、换个思路来分配

Mar By 苏剑林 | 2025-03-05 | 23972位读者

这篇文章我们继续探讨MoE的负载均衡问题。在上一篇文章《MoE环游记：2、不患寡而患不均》中，我们主要讨论了通过Aux Loss来促进负载均衡的思路。Aux Loss固然简单直观，但它也有一个明显的缺点——权重不好调——调低了无法促进均衡，调高了容易损害LM Loss，所以业界一直有寻找替代方案的尝试。

本文要分享的是名为“Loss-Free”的方案，由DeepSeek在《Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts》提出。和DeepSeek众多耀眼的开源作品相比，这篇论文也许不算起眼，但在笔者看来，它潜在的学术影响力可能远超其他工作，因为所提方法不仅简单有效，而且极具普适性，堪称经典。

方法大意

面对负载不均衡，Aux Loss的应对思路是通过额外的损失引导Router给出均衡的打分，而Loss-Free的想法则是换个新的分配思路，即不改变Router现有打分结果，而是改变 $\text{argtop}_k \rho$ 这个分配方式。

其实这个方向此前也有过一些努力。比如2021年Facebook提出了BASE Layer，将Expert的分配视为线性指派问题，即以负载均衡为约束条件，求在该约束之下Router总打分尽可能高的分配结果，这可以用匈牙利算法等来解决。但该方案需要知道全体Token的打分，所以对于自回归式LLM来说，它只适用于训练，推理还是只能用 $\text{argtop}_k \rho$ ，训练推理存在不一致性，并且由于目前求解算法的限制，它只适用于 $k=1$ 的场景。

相比之下，Loss-Free的做法非常简单且有效，它留意到一个事实，即我们总可以引入一个偏置项 b ，使得 $\text{argtop}_k \rho + b$ 的分配是均衡的，所以它将MoE的形式改为

$$y = \sum_{i \in \text{argtop}_k \rho} \rho_i e_i \quad \rightarrow \quad y = \sum_{i \in \text{argtop}_k \rho + b} \rho_i e_i \quad (1)$$

这里的 b 是输入无关的向量，由训练过程确定下来，训练完后它就保持不变，因此推理阶段也可以用，换言之训练和推理具有一致的形式。注意乘以 e_i 的还是 ρ_i 而不是 $\rho_i + b_i$ ，也就是说 b 仅仅参与分配过程而不参与MoE的前向计算，所以我们对 b 或 $\rho + b$ 的正负性都没有特殊要求。

手搓梯度

怎么训练 b 呢？我们知道， b 的优化方向自然是促进负载均衡，为此按照上一篇的记号，我们先定义 $f = [f_1, f_2, \dots, f_n]$ ：

$$f_i = \begin{cases} 1/k, & i \in \text{argtop}_k \rho + b \\ 0, & i \notin \text{argtop}_k \rho + b \end{cases} \quad (2)$$

以及 $F = \mathbb{E}[f]$ ，这里的 F 自然就是在 b 偏置下Expert当前的负载分布了。借着我们定义均匀分布为 $Q = (1/n, 1/n, \dots, 1/n)$ ，那么负载均衡就相当于最小化

$$\mathcal{L}_{\text{aux}} = \frac{1}{2} \|F - Q\|^2 = \frac{1}{2} \sum_{i=1}^n (F_i - 1/n)^2 \quad (3)$$

这个目标是不可导的，但有了上一篇的经验，我们知道STE (Straight-Through Estimator) 可以解决这个问题

。STE的关键是找一个可导且跟 $\mathbf{F}$ 具有同增减趋势的量作为 $\mathbf{F}$ 的光滑近似，这里我们的优化参数只有 $\mathbf{b}$ ，而它正好具有我们期望的性质（增大 b_i ， i 被选中的概率就更高，那么 F_i 就更大），所以答案就呼之欲出了：

$$\mathcal{L}_{\text{aux}} = \frac{1}{2} \|\mathbf{b} + \text{sg}[\mathbf{F} - \mathbf{b}] - \mathbf{Q}\|^2 = \frac{1}{2} \sum_{i=1}^n (b_i + \text{sg}[F_i - b_i] - 1/n)^2 \quad (4)$$

它的梯度是

$$\nabla_{\mathbf{b}} \mathcal{L}_{\text{aux}} = \frac{1}{2} \nabla_{\mathbf{b}} \|\mathbf{b} + \text{sg}[\mathbf{F} - \mathbf{b}] - \mathbf{Q}\|^2 = \mathbf{F} - \mathbf{Q} \quad (5)$$

所以用梯度下降（SGD）来更新 $\mathbf{b}$ 就是

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha(\mathbf{F} - \mathbf{Q}) \quad (6)$$

这里 α 是 $\mathbf{b}$ 的学习率。不过Loss-Free最终选择的更新规则略有不同，它选择的是符号梯度下降（SignSGD）：

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \text{sign}(\mathbf{F} - \mathbf{Q}) \quad (7)$$

这个结果其实也很好理解，就是如果 F_i 比 $1/n$ 大，那么就调小一点 b_i ，否则就增大一点 b_i 。

一脉相承

原论文在介绍Loss-Free时，并没有上述Aux Loss的推导过程，而是直接给出式(7)的更新规则，给人的感觉是给 $\mathbf{b}$ “手搓”了梯度 $\text{sign}(\mathbf{F} - \mathbf{Q})$ ，这也是它Loss-Free这个名字的来源。

然而，从本文给出的推导可以看出，更新规则(7)也完全可以从Aux Loss视角得到，两者是一脉相承的。看起来Loss-Free最直接的好处是不用调Aux Loss权重了，但它实际上也有个学习率参数 α 要调，尽管原论文已经帮我们搜好 $\alpha = 0.001$ 这个默认值，但不可否认这个超参数是存在的。

在笔者看来，Loss-Free的本质创新并不是没有Aux Loss，而是隔离了Aux Loss和LM Loss的优化参数，从而达到了负载均衡和模型能力两不误的效果。其中最关键一步，是留意到“一个偏置项足以达到负载均衡”这一事实，然后就让Aux Loss只优化新引入的偏置 $\mathbf{b}$ ，而LM Loss则优化剩余参数，让Aux Loss对LM Loss的负面作用降到最低。

相比之下，常规的Aux Loss方案需要全体参数来促进负载均衡，而LM Loss优化的也是全体参数，两者的优化方向可能并不完全兼容，因此想找到一个最优的平衡点相对来说就更为困难。所以，Loss-Free基于“一个偏置项足以达到负载均衡”将两个Loss的优化参数隔离开来，是负载均衡问题的一个绝妙的解决办法。

相关细节

尽管Loss-Free已经足够简单明了，但是在使用的时候还要稍微注意一些细节。

首先，对于每个Batch的数据，我们应当先根据LM Loss来更新模型参数，然后再根据式(7)来更新 $\mathbf{b}$ 。这是因为 $\mathbf{b}$ 的更新依赖于全体Token的统计信息 $\mathbf{F}$ ，先更新 $\mathbf{b}$ 再更新模型其余参数的话，原则上会有泄漏未来信息的风险。虽然直观看来就一个向量 $\mathbf{b}$ 泄漏不了多少信息，但这个风险终究是存在的，因此要尽量去规避它。

其次，刚才我们说原论文已经调好 $\alpha = 0.001$ ，但这个结果可能跟原论文用Sigmoid作为Router ρ 激活函数的选择是绑定的。原因也不难想，经过Sigmoid后，每个 ρ_i 相对比较独立，并且都在 $(0, 1)$ 内， $\alpha = 0.001$ 相当于说

每一步的更新幅度约为千分之一，如果换Softmax、ReLU或者其他激活函数，那么就可能需要重调 α 了。

针对这个问题，笔者建议的做法是结构Gate和Bias所用的激活函数，即

$$\mathbf{y} = \sum_{i \in \text{argtop}_k \boldsymbol{\rho} + \mathbf{b}} \rho_i \mathbf{e}_i \rightarrow \mathbf{y} = \sum_{i \in \text{argtop}_k \boldsymbol{\rho}^{(\sigma)} + \mathbf{b}} \rho_i^{(h)} \mathbf{e}_i \quad (8)$$

其中 $\boldsymbol{\rho}^{(\sigma)} = \sigma(\mathbf{x}\mathbf{W}^{(R)})$, $\boldsymbol{\rho}^{(h)} = h(\mathbf{x}\mathbf{W}^{(R)})$, $\sigma(\cdot)$ 是Sigmoid函数, $h(\cdot)$ 是任意单调且值域非负的函数, 说白了就是加上 $\mathbf{b}$ 的是Sigmoid激活的打分, 这样我们就可以复用 $\alpha = 0.001$, 至于乘上Expert的Gate, 我们可以用其他激活函数, 只要它的单调性跟Sigmoid一致就行。

此外, 由于更新规则(7)加了sign函数, 因此有可能训出绝对值大于1的 b_i , 整体绝对值还可能越来越大, 这些都是正常的, 对模型效果不会有影响。实际上 $\mathbf{b}$ 有一个冗余的自由度, 因为全体 b_i 都加上同一个常数后, $\text{argtop}_k \boldsymbol{\rho} + \mathbf{b}$ 的结果不变。这个额外的自由度我们可以用来做其他好玩的事情 (且听下回分解)。

延伸思考

除了MoE的负载均衡之外, Loss-Free的思想还可以应用到很多类似问题, 比如VQ-VQE的编码表坍塌 (Codebook Collapse), 就可以用同样思路解决, 而且相比之前介绍的“[旋转技巧](#)”、“[线性变换技巧](#)”显得更自然和普适。事实上, 本文开篇的评价“Loss-Free潜在的学术影响力可能远超其他工作”, 正是基于Loss-Free的普适性考虑的。

抛开具体的应用背景, 从数学上来看, Loss-Free的贡献可以理解为给出了用梯度下降来求解指派问题的方法。一个经典的线性指派问题可以表示为:

$$\min_f \sum_{i=1}^n c_{i,f(i)} \quad (9)$$

其中 $c_{i,j}$ 是给定的成本函数, f 是 $\{1, 2, \dots, n\}$ 到自身的双射。放到本文的背景下, $c_{i,j}$ 不就相当于 n 个Token、 n 个Expert的打分, 所求 f 不就是一个负载均衡的分配方案? 求解此类问题的一般想法是在满足约束条件的空间里搜索尽可能优的解, 而Loss-Free则反过来, 先构建一个最优但不一定满足约束条件的解:

$$f(i) = \underset{j}{\operatorname{argmin}} c_{i,j} \quad (10)$$

这个解在分数上肯定是最优的, 但不一定满足双射的条件, 这里不满足双射就等价于负载不均衡。于是我们引入偏置

$$f(i) = \underset{j}{\operatorname{argmin}} c_{i,j} + b_j \quad (11)$$

b_j 初始化为零, 然后根据式(7)来更新, 更新规则说白了就是哪个 j 出现次数多, 那减少相应的 b_j , 反之增加, 直到出现双射为止。

文章小结

本文介绍了MoE负载均衡问题的Loss-Free方法, 它由DeepSeek提出, 其核心在于通过引入一个简单的偏置项来实现负载均衡。本文进一步思考了它与Aux Loss的联系, 以及它在类似数学问题上的应用潜力。

转载到请包括本文地址：<https://spaces.ac.cn/archives/10757>

更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (Mar. 05, 2025). 《MoE环游记：3、换个思路来分配》 [Blog post]. Retrieved from <https://spaces.ac.cn/archives/10757>

```
@online{kexuefm-10757,  
  title={MoE环游记：3、换个思路来分配},  
  author={苏剑林},  
  year={2025},  
  month={Mar},  
  url={\url{https://spaces.ac.cn/archives/10757}},  
}
```